

Fábrica de Software

Arquitetura e MVP • versão 2.1

Assinaturas de IA e clientes oficiais
Controle e execução na mesma VPS

Vinícius • 29 de setembro de 2026

Documento integral de planejamento. Inclui arquitetura, funcionamento, economia, contratos, sprints, guias Claude/Codex/Antigravity e templates de documentação. Nenhum software foi implementado.

Conteúdo

- 01. README.md
 - 02. documentacoes/arquitetura/ARQUITETURA.md
 - 03. documentacoes/infraestrutura/DIMENSIONAMENTO-VPS.md
 - 04. documentacoes/arquitetura/ADR-002-vps-unica.md
 - 05. PLANO-MVP.md
 - 06. ESPEC-MVP.md
 - 07. documentacoes/runtime/README.md
 - 08. documentacoes/handoff/README.md
 - 09. documentacoes/operacao/README.md
 - 10. documentacoes/economia/README.md
 - 11. documentacoes/POLITICA-IA.md
 - 12. AGENTS.md
 - 13. CLAUDE.md
 - 14. ANTIGRAVITY.md
 - 15. PILOTO.md
 - 16. BACKLOG.md
 - 17. MATRIZ-COBERTURA.md
 - 18. FONTES.md
 - 19. PROMPT-INICIAL.md
 - 20. templates/TICKET.md
 - 21. templates/ENTREGA.md
 - 22. templates/HANDOFF.md
 - 23. templates/LESSON.md
 - 24. documentacoes/INDEX.md
 - 25. lessons/INDEX.md
 - 26. CHANGELOG.md
 - 27. .agents/rules/documentacao.md
- Anexo final: config/policies.example.json

Fábrica de Software — kit v2.1

29/09/2026. Planejamento completo: web na VPS, worker interno na VPS, clientes oficiais e assinaturas primeiro. APIs e créditos extras desligados no MVP. Não há implementação neste pacote.

Começar

1. Ler documentacoes/arquitetura/ARQUITETURA.md, FONTES.md e PLANO-MVP.md.
2. Preencher PILOTO.md e executar FAC-001/FAC-002 antes de bootstrap completo.
3. Mesclar AGENTS.md e CLAUDE.md com regras existentes; preservar escopos locais.
4. Antigravity: ler ANTIGRAVITY.md e confirmar como a versão carrega regras; injetar explicitamente quando necessário.
5. Seguir documentacoes/POLITICA-IA.md em toda entrega; usar templates.

Conteúdo

Arquitetura, contratos, backlog, políticas, guias dos três agentes, runtime, handoff, operação, economia, templates e matriz de migração. Configuração example é política da fábrica a implementar, não configuração nativa dos fornecedores.

O PDF reúne todos os Markdown deste kit, inclusive guias e templates. Markdown é fonte editável; ao atualizar, regenerar PDF a partir da mesma revisão. O pacote preserva a cobertura do kit anterior, substituindo decisões incompatíveis com a v2.

Infraestrutura do MVP

Toda a fábrica executa na mesma VPS. Bom recomendado: 8 vCPU, 16 GB RAM, 200 GB SSD/NVMe, um executor inicial. Ler documentacoes/infraestrutura/DIMENSIONAMENTO-VPS.md e ADR-002; mínimo/ideal e condições de escala documentados. Revisão 2.1 mantém os nomes dos arquivos para continuidade.

Software Factory - Arquitetura e operação

Versão 2.1 - 2026-09-29

1. Decisão e escopo v2.1

Decisão do usuário em 29/09/2026: toda a fábrica na mesma VPS Linux, incluindo painel/API, PostgreSQL, Redis, supervisor, clientes oficiais, autenticação, sandboxes, worktrees, builds, testes e documentação. MacBook não é necessário para executar o MVP. Assinaturas primeiro; APIs e créditos extras desativados. Nenhum serviço foi implementado nesta entrega.

Stack proposta preservada: Laravel, Angular, PostgreSQL, Redis e supervisor Node/TypeScript. Usar módulos/processos separados, não exigir microserviços. O software piloto mantém sua stack.

Inferência permanece nos fornecedores por internet, sem GPU ou modelo local na VPS. Autenticar cada cliente pelo fluxo oficial na identidade de execução dedicada; validar headless, plano, compatibilidade e cobrança no ambiente real.

Uso pessoal de desenvolvimento. Multiusuário comercial, revenda e compartilhamento de credenciais ficam fora do MVP e exigem avaliação das condições aplicáveis.

2. Arquitetura numa VPS

Navegador -> HTTPS/proxy -> Angular/Laravel -> Orchestrator/Context Builder -> PostgreSQL/outbox/Redis -> worker interno -> Agent Runtime Gateway -> Codex/Claude/Antigravity oficiais -> sandbox/worktree -> checks/review/docs -> painel/aceite.

Separar serviços de controle, supervisor confiável, cliente de IA e execução de código por identidades, redes e volumes. Redis/PostgreSQL internos, sem portas públicas. Worker usa protocolo interno autenticado para claim/heartbeat/events/checkpoint/complete; preservar contrato que permite execução remota futura sem torná-la requisito.

Credenciais ficam nos mecanismos oficiais do cliente na VPS, fora do Git, logs, artefatos, contexto e mounts do código do piloto. A fábrica não coleta senhas nem reutiliza token OAuth como API. Login humano inicial e renovação quando solicitada; não prometer sessão permanente.

Agent Runtime Gateway gerencia lifecycle e resultados. CLI pode executar ferramentas autonomamente, exigindo limites no host. Validar que código/testes não leem credenciais; se isso não for possível no modo instalado, bloquear adapter e registrar ADR de isolamento compatível.

Um writer por worktree, revisão em sessão independente e revisão exata. Controle/executor compartilham host: falha da VPS afeta ambos. Backups ficam fora dela. Modelo cloud não transforma sandbox em isolamento de VM.

3. Dimensionamento e operação

Perfis de engenharia para este MVP, não requisitos oficiais de fornecedores nem benchmark. Detalhes em documentacoes/infraestrutura/DIMENSIONAMENTO-VPS.md.

Perfil	vCPU	RAM	SSD/NVMe útil	Execução inicial
Mínimo	4	8 GB	120 GB	1 job leve, etapas sequenciais
Bom - recomendado para começar	8	16 GB	200 GB	1 job completo, testes e browser sequenciais
Ideal para amadurecer o MVP	8	32 GB	300 GB	1 job inicial; até 2 em worktrees distintos após ensaio

Linux 64 bits mantido, preferir x86_64 para reduzir incerteza de binários; ARM exige prova de compatibilidade. Sem GPU. CPU sustentada e I/O importam; vCPU compartilhada não equivale a núcleo dedicado. Disco precisa acomodar SO, banco, imagens, caches, worktrees e artefatos com 20% de margem livre.

No perfil Bom, reservar envelope inicial de 6 GB para SO/controle/banco/cache/supervisor; até 6 GB para execução de um job, incluindo seus processos; manter 4 GB de margem. Limite agregado precisa ser imposto via cgroups/supervisor, não só por container individual.

Começar sempre com MAX_CONCURRENT_EXECUTORS=1. Não executar build pesado, review editável ou E2E em paralelo por padrão. Promover concorrência só após medir picos, latência do painel, OOM, swap/I/O e disco.

Pausar novos jobs com disco abaixo de 20% ou memória sustentada acima de 85%. Abort/timeout conservador, preservar checkpoint e logs limitados. Worker parado gera WAITING_WORKER; reboot/rede/serviço são cenários de recuperação, não dependência de laptop doméstico.

Backup diário PostgreSQL criptografado fora da VPS; proposta sete diários/quatro semanais. Git remoto e artefatos relevantes externos. RPO 24h/RTO 4h são metas até restauração comprovada. Retenção inicial artefatos 30 dias/auditoria resumida 90 dias; limpeza sem atingir volumes persistentes.

Mensalidade/renovação, backup, armazenamento externo, assinaturas, impostos/câmbio e manutenção compõem TCO. Energia do servidor normalmente integra a hospedagem; não somar energia de worker doméstico nesta arquitetura.

4. Funcionamento de um ticket

Entrada obrigatória: objetivo, comportamento atual e esperado, critérios de aceite verificáveis, escopo permitido, caminhos proibidos, comandos de teste e limites de execução e gasto. Tickets vagos retornam à triagem antes de gastar com implementação.

Estados: DRAFT -> READY -> RUNNING -> VALIDATING -> REVIEW -> DOCS -> AWAITING_HUMAN -> DONE. WAITING_WORKER, WAITING_PROVIDER, PAUSED_LIMIT, PAUSED_BUDGET, BLOCKED e FAILED são estados explícitos; cancelamento interrompe novas chamadas e encerra o sandbox com evidências preservadas.

1. Inspeção determinística: branch base e SHA, instruções do repositório, dependências, testes existentes e arquivos relevantes. 2. Plano curto de implementação. 3. Cliente oficial implementa no workspace autorizado. 4. Supervisor impõe isolamento/lifecycle e coleta resultados; comandos internos do CLI não são interceptados apenas pelo prompt. 5. Lint/testes/scanners. 6. Revisão independente de diff e resultados. 7. Correções limitadas. 8. Documentação. 9. Diff/PR para aceite humano.

Revisor pode usar o mesmo provedor em sessão independente; isso não assegura ausência de vieses compartilhados. Segurança começa com ferramentas determinísticas e recebe revisão

especializada conforme risco.

Cada job guarda ticket_id, run_id, attempt_id, base_sha, etapa, lease e heartbeat. Transição usa controle otimista ou lock transacional. Lease expirado recupera o job; side effects exigem idempotência. Uma chamada de IA com resultado desconhecido não deve ser reexecutada automaticamente sem reconciliar efeitos e possível consumo duplicado de cota.

No piloto, produzir branch e diff local é suficiente. Publicação de PR exige configurar a integração. Merge e deploy continuam manuais. Um gate documental pode aprovar arquivos estruturalmente válidos sem assegurar a correção técnica do conteúdo; a revisão deve conferi-los.

5. Agentes e responsabilidades

Planner/Tech Lead: decompor objetivo, classificar risco e selecionar estratégia. Developer: implementar dentro do escopo. Reviewer: procurar falhas de comportamento, regressões e incoerências no diff. QA: relacionar critérios de aceite a evidências. Documentation: registrar somente o que foi efetivamente implementado. Security: revisar achados e áreas sensíveis.

Os papéis são perfis de instrução e tarefas, não seis servidores nem seis chamadas obrigatórias a cada ticket. No MVP, usar Developer e Reviewer; o executor realiza verificações e a documentação faz parte da entrega. Planejamento extenso, QA separado e segurança por IA entram quando necessários.

R0: documentação e alterações cosméticas. R1: lógica local e validações comuns. R2: integrações e mudanças em mais de um domínio. R3: autenticação, autorização, pagamentos, segredos ou dados pessoais. R4: produção, infraestrutura e migrações irreversíveis. R3/R4 não pertencem ao primeiro piloto.

Ferramentas expostas: leitura limitada, busca, patch, comandos de teste cadastrados e diff. O modelo não recebe shell irrestrito, credenciais de produção ou Docker socket. Instruções encontradas em arquivos, logs e páginas são dados não confiáveis quando solicitam ultrapassar a política do job.

6. Context Builder e economia

Primeiro reduzir trabalho, depois escolher modelo. Ticket pequeno, critério claro e baseline de testes evitam ciclos caros. Não enviar todo o repositório, node_modules, vendor, binários, dumps ou logs integrais.

TicketContext contém: objetivo, critérios, regras aplicáveis, trechos da especificação, ADR relevante, arquivos selecionados, testes e resumo dos achados. Registrar hashes e base_sha. Começar com orçamento de 20-40 mil tokens de entrada por chamada; adaptar ao modelo e tarefa. É um teto proposto, não uma necessidade mínima.

Seleção inicial por caminhos, imports, dependências e busca textual. Indexação semântica/embeddings só após demonstrar benefício. Resumo não substitui fonte: incluir referências e permitir abrir o trecho original. Configurar limites de tamanho e sinalizar truncamento.

Gates baratos primeiro: lint, testes, type check, secret scan e dependency scan conforme stack. Fornecer à IA apenas os achados úteis, diff e contexto. Evitar pedir a um modelo que redescubra o que uma ferramenta detecta diretamente.

Prefixo estável pode favorecer cache administrado pelo cliente quando suportado; a análise financeira seguinte aplica-se somente à extensão futura de APIs. Não assumir que qualquer trecho

repetido vira cache ou que cache funciona entre provedores/modelos. Medir tokens efetivamente em cache. Não omitir invalidação de regras ou contexto para preservar desconto.

Como extensão futura de API, para relatórios e auditorias não urgentes, avaliar Batch, observando elegibilidade, SLA e tarifas. Não colocar uma etapa interativa bloqueante em lote apenas pelo desconto. Batch e cache têm condições próprias; não somar descontos sem conferir a tabela do provedor.

Respostas estruturadas e patches reduzem saída desnecessária. Limitar raciocínio quando houver controle suportado, sem cortar capacidade necessária. Caching de resultados internos só para entradas equivalentes e mesma revisão; nunca reutilizar aprovação de diff alterado.

7. Agent Router, cotas e limites

Seleção por capacidade exigida, elegibilidade do modo de autenticação, disponibilidade do worker, saúde do cliente, cota observada, risco e desempenho medido. Preferências Claude para análise/review, Codex para implementação e Antigraity para UI/QA são hipóteses iniciais; não representam ranking comprovado.

Estados do provider: UNCONFIGURED, AUTH_REQUIRED, AVAILABLE, BUSY, RATE_LIMITED, COOLDOWN, ERROR e DISABLED. Estado de uso inclui value nullable, unidade, fonte, observed_at e reset_at nullable. Saldo desconhecido não significa disponível ilimitadamente; o painel mostra desconhecido. Não inventar percentuais ou horários de reset.

Uma assinatura pode ter várias janelas e limites por modelo/modo. O adapter deve guardar as restrições distintas. AVAILABLE significa último teste elegível, não garantia de próxima chamada. Reativar após horário informado com uma sondagem barata; se horário ausente, backoff limitado e verificação posterior.

Atingiu cota: parar writer, preservar checkpoint, registrar motivo e escolher outro provider elegível. Se nenhum disponível, WAITING_PROVIDER. Não criar contas, alternar identidades ou usar endpoints privados para contornar limites. Trocar entre assinaturas próprias não aumenta a cota de nenhuma delas.

Limites propostos: duas rodadas de correção, uma escalada, dois handoffs por ticket e 30 minutos por tentativa. Falha repetida idêntica pausa com diagnóstico. Auth exige ação humana no cliente oficial; 5xx transitório tem retry limitado; limite de uso não recebe retry imediato.

UsageMeter guarda métricas realmente expostas: tokens/modelo/tempo/ferramentas, quando disponíveis. Custo teórico de tokens apresentado pelo CLI não equivale a cobrança de assinatura. Ledger distingue assinatura fixa, extra medido, API estimada/reconciliada e desconhecido.

MVP: monthly_api_budget=0, fallback_api.enabled=false, créditos/extra usage/autorecharge desativados em cada conta. Bloquear também variáveis de API key herdadas no processo. Configuração da fábrica não controla cobrança habilitada no site do fornecedor: o preflight precisa verificá-la com o usuário no onboarding.

BudgetGuard monetário permanece preparado para futura API, com reservas atômicas e reconciliação; não é usado para fingir reservar tokens de assinatura sem suporte do fornecedor. RuntimeGuard aplica tempo, tentativas, processos, concorrência e limites de troca.

8. Economia e comparação financeira

Retirar a simulação por tokens da arquitetura principal: o modelo agora é assinatura/capacidade. Não prometer quantidade de tickets por plano nem economia percentual antes do experimento.

Custo mensal = VPS + backup + assinaturas + extras autorizados + armazenamento externo + manutenção. Custo incremental = novas despesas causadas pela fábrica. Custo por aceite = custo atribuído ao experimento / número de tickets aceitos, incluindo falhas no numerador. Quando não houver aceitos, informar indefinido.

Para atribuir custo fixo compartilhado, registrar método: por exemplo fração do tempo de uso ou alocação definida pelo responsável. Não somar preço teórico do CLI à assinatura como se fosse cobrança real. Mostrar separadamente custo total e incremental.

Economia principal: reaproveitar planos existentes; poucos papéis por ticket; reduzir contexto; ferramentas determinísticas antes de IA; instruções estáveis; tickets pequenos; handoffs curtos; revisão proporcional ao risco; evitar loops e paralelismo que esgotam cota.

Cache administrado pelo cliente pode ajudar uso/latência, mas não assumimos desconto em assinatura. Batch permanece extensão de API futura, não requisito nem desconto disponível nos CLIs. Modelos econômicos devem ser selecionados apenas quando expostos e elegíveis no plano.

Quando todas as cotas acabarem, esperar é o padrão com gasto adicional zero autorizado. APIs futuras só após mudança explícita de política, configuração de credencial, tarifas oficiais e limites de cobrança. A ausência de API não impede consumo de créditos extras do CLI se o usuário os habilitar: por isso o onboarding verifica ambos.

Comparar 10 tickets pequenos: aceite, regressões, tempo ativo, espera por cota, correções, handoffs, tokens quando observáveis e custo atribuído. A economia só estará demonstrada quando confrontada com baseline equivalente.

9. Isolamento e confiabilidade

Sandbox é workspace descartável com usuário sem privilégios, limites de CPU/memória/processos/tempo/disco, filesystem mínimo, capabilities reduzidas e rede restrita. Dependências podem exigir saída à internet via destinos controlados; o sandbox não deve alcançar banco/Redis da fábrica ou serviços internos.

Não montar /var/run/docker.sock no sandbox; não usar privileged; não montar pasta de segredos. O Executor Manager confiável prepara ambientes. Se o projeto requer containers de integração, usar instâncias pré-provisionadas ou mecanismo isolado definido em ADR; não dar controle do host ao agente.

No worker, containers compartilham kernel e uma falha do host afeta controle e execução. Restrição de recursos reduz interferência, não elimina risco de escape. O piloto usa código de um repositório próprio/confiável e dados sintéticos. Execução de código arbitrário de terceiros exige reavaliar o isolamento.

Revisão humana permanece para merge/deploy, mudança de credenciais e operações destrutivas. Logs não devem vaziar PII ou segredos. Remover segredos antes de enviar arquivos aos provedores; avaliar retenção e uso de dados da modalidade contratada escolhida.

Recuperação: reiniciar executor em teste, comprovar lease expirado e recuperação sem duplicar efeitos; pausar ao exceder orçamento; testar falha do cliente, cota indisponível e timeout; restaurar backup. Nunca marcar DONE apenas porque o modelo declarou sucesso.

10. MVP incremental

V0: preencher piloto, validar os três clientes no ambiente real, confirmar autenticação e cobrança, executar três tickets assistidos e um handoff manual. Validar um provider por vez; não bloquear

primeiro teste porque os três ainda não estão prontos.

V1: controle web, worker por HTTPS, lease/heartbeat, um adapter oficial, sandbox, Context Builder, Developer, checks e gate documental. Nenhuma API paga. Revisor em sessão independente, com revisão humana final.

V2: segundo adapter, Provider Manager e handoff automático após writer encerrado, ensaio de falha/cota e dez tickets. Terceiro adapter pode entrar após prova de compatibilidade. UI/QA só exige browser se o critério precisar dele.

V3: expandir projetos, workers ou workflow engine quando métricas justificarem. Temporal, autoscaling, APIs e inferência local seguem como opções futuras.

Fora MVP: merge/deploy automático, multiusuário comercial, credenciais compartilhadas, GPUs, execução de código arbitrário de terceiros e features de risco R3/R4.

Metas propostas: 8/10 aceitos em até duas correções; 100% com docs e evidências; zero extras/API não autorizados; zero writers simultâneos no mesmo worktree; recuperação de interrupção comprovada. Não são resultados medidos.

11. Documentação como parte da entrega

Toda feature, correção, refatoração, configuração ou migração exige relatório datado em documentacoes/<dominio>/AAAA-MM-DD-TICKET-titulo.md. O relatório registra objetivo, funcionamento, alterações, diff sanitizado, testes e resultados reais, riscos, rollback e limitações.

Além do histórico, atualizar a documentação de estado atual:

documentacoes/<dominio>/README.md, arquitetura/ADRs quando necessário, contratos/API, índice, CHANGELOG e status do backlog. Não basta acumular relatos de entrega e deixar a descrição atual obsoleta.

Lessons: criar ou atualizar lessons/<conceito>.md com explicação, motivo, exemplo real, cuidados e links. Reutilizar documento existente; não inventar aprendizado ou gerar cópias por ticket.

Atualizar o índice.

Documento de entrega vincula base_sha/head_sha ou diff de uma revisão identificada. Não tentar incluir no próprio documento o hash do commit que ainda vai conter esse documento: usar revisão do código anterior ao commit documental ou link final do PR. Remover segredos e dumps dos diffs.

Definition of Done: critérios verificados, checks relevantes aprovados, revisão resolvida, custo reconciliado ou explicitamente pendente, documentação técnica atualizada, relatório de entrega, lessons quando aplicável, diff revisável e aceite humano. Gate de CI exige arquivos e seções; revisão humana avalia a veracidade.

Os arquivos AGENTS.md, CLAUDE.md e .agents/rules/documentacao.md remetem à política comum. Antigravity exige confirmar que a versão instalada lê a pasta de regras; se não, inserir a política explicitamente no contexto. Esses MD são orientações de repositório, não instalação de skills ou garantia de leitura automática.

12. Contratos e painel

Detalhamento em ESPEC-MVP.md, documentacoes/runtime/README.md, documentacoes/handoff/README.md e documentacoes/operacao/README.md. Entidades: Project, Ticket, Run, Step, Worker, ProviderInstallation, UsageObservation, AgentExecution, Checkpoint, ContextManifest, Artifact, LedgerEntry e Approval.

Runtime request: ticket/run/step/attempt, workspace e revisão, provider/modelo solicitado, prompt/context manifest, ferramentas e limites, regras documentais. Result: status normalizado, exit code, revisão/diff, artefatos/checks, uso nullable, versão do cliente, modelo efetivo e checkpoints. Nunca armazenar cadeia de raciocínio privada.

Interface: execute, getStatus, getCapabilities, getUsage, cancel, resume. Capacidade ausente deve retornar unsupported, não simular equivalência. Resume nativo só no mesmo provider/versão compatível; troca de provider é nova execução com handoff. Cancel deve encerrar árvore de processos e confirmar quiescência.

Painel mobile: tickets, etapa, writer, worker online/offline, health dos providers, última observação de uso e sua fonte, espera por cota, tentativas, diff, checks e docs. Percentual de uso apenas se fornecedor expõe e adapter validou. API/extra desligados visíveis. Aceite associado a revisão exata; novo código invalida aprovação.

CLI encerrou com zero não prova sucesso do ticket. Validar schemas, diff, checks, critérios e documentação antes de AWAITING_HUMAN. Publicação de PR é integração opcional autenticada com escopo mínimo.

13. Fontes e pendências

Fontes oficiais verificadas em 29/09/2026, listadas integralmente em FONTES.md. Confirmado: Codex possui exec não interativo e reutiliza autenticação do CLI; Claude Code possui -p; Google documenta agy -p para automação. Isso comprova mecanismos técnicos, não capacidade ilimitada ou elegibilidade automática de qualquer uso.

Autenticação oficial, plano contratado, modo de execução, créditos extras, termos e compatibilidade precisam ser validados no worker antes de ativar o adapter. No Claude, diferenciar cliente oficial de SDK/serviço usando credenciais de assinatura; não capturar/intermediar tokens OAuth. Documentação de assinatura não substitui análise de modo de cobrança real.

Pendências para execução: repo/branch, feature piloto, baseline, perfil contratado da VPS e compatibilidade dos clientes, assinaturas e modelos acessíveis, forma oficial de login, VPS e capacidade semanal. Stack da fábrica proposta pode ser confirmada antes de bootstrap. O planejamento avança sem inventar estas respostas.

Esta v2.1 substitui a topologia VPS + MacBook da v2 e a execução por APIs da v1. Mantém qualidade, isolamento, documentação por domínio, lessons, evidências, backlog e aceite. Nenhum cliente foi instalado ou testado nesta entrega.

Dimensionamento da VPS única - MVP v2.1

Status: PLANEJADO. Estimativa de engenharia, sujeita a ensaio do repositório. Não são requisitos oficiais dos CLIs. Uma VPS abriga controle, banco/fila, clientes, execução e checks; inferência nos fornecedores, sem GPU.

Perfis

Recurso	Mínimo	Bom - recomendação inicial	Ideal para o MVP
CPU	4 vCPU	8 vCPU	8 vCPU; ampliar se CPU for gargalo
RAM	8 GB	16 GB	32 GB
Disco útil provisionado	120 GB SSD	200 GB SSD/NVMe	300 GB SSD/NVMe
Jobs de código simultâneos	1 leve	1 completo	1 inicialmente; até 2 validados
Browser/E2E	Opcional, leve e sequencial	1 sessão sequencial	1-2 conforme medição
Projetos ativos no experimento	1	1	1; expandir depois

Todos: Linux 64 bits suportado/atualizado, preferência x86_64; compatibilidade dos binários confirmada; Docker/Compose quando usados; Git/Node/PHP e toolchains do piloto; HTTPS, saída aos fornecedores/Git/dependências; backup externo e Git remoto. Tráfego depende de imagens, builds e artefatos; medir antes de contratar franquia. GUI completa e GPU não são requisitos; browser headless só quando necessário.

Mínimo: 4 vCPU / 8 GB / 120 GB

Viável como ponto de partida para repositório pequeno, poucos serviços e build leve. Envelope inicial: SO/controle/Postgres/Redis/supervisor até 3 GB; execução agregada até 3 GB; margem 2 GB. Limite do job 2 vCPU/3 GB, 256 processos e 30 min, ajustáveis após ensaio. Limitar serviços de integração e não rodar builds e E2E simultâneos.

Não representa garantia de que qualquer Angular/Laravel/build caberá. Se baseline exceder limite, usar Bom antes do experimento completo. Swap pode amortecer picos, mas não substitui RAM; evitar execução sustentada em swap. Não selecionar VPS de 2 GB/4 GB para este escopo completo com builds.

Bom: 8 vCPU / 16 GB / 200 GB

Perfil recomendado para começar o MVP com folga: controle e um job, incluindo build/testes; browser e revisão sequenciais. Envelope: SO 1.5 GB, controle/API/scheduler 1.5 GB, PostgreSQL 1.5 GB, Redis 0.5 GB, supervisor/observabilidade 1 GB = 6 GB; execução agregada até 6 GB; margem 4 GB.

São envelopes de planejamento, não configurações finais de PostgreSQL/PHP. Guardar orçamento global com overhead/cache do kernel; medir RSS e memória do cgroup. Limite inicial job 4 vCPU/6 GB/256 processos/30 min. Capacidade de review e dependências auxiliares precisa caber no mesmo envelope se ativa na etapa.

Ideal: 8 vCPU / 32 GB / 300 GB

Prioriza margem de memória para builds, serviços de integração e QA. Envelope: host/controle até 8 GB; até dois jobs com 8 GB cada; margem 8 GB. Inicialmente apenas um job, limite 4 vCPU/8 GB. Ao testar dois, limitar cada um a aproximadamente 3 vCPU e validar latência/I/O, sem dois writers no mesmo worktree.

Mais RAM não cria cota de IA. Dois jobs podem disputar CPU, disco e cotas; se CPU sustentada for o gargalo, avaliar 12-16 vCPU ou CPU dedicada apenas após medir. Ideal significa conforto para este MVP, não alta disponibilidade nem tamanho final para múltiplos produtos.

Disco e retenção

Bom, exemplo de orçamento dos 200 GB: SO/ferramentas 25 GB; imagens/caches 55 GB; worktrees/dependências temporárias 45 GB; dados persistentes 15 GB; logs/artefatos temporários 20 GB; margem livre 40 GB. Valores dependem do projeto; limitar logs, cache e número de workspaces.

Disco anunciado não é todo livre após instalação; confirmar espaço útil. Backup no mesmo disco não protege falha da VPS. Limpar apenas jobs concluídos e evidências já preservadas. Não usar prune indiscriminado nem apagar volumes persistentes.

Condições de contratação

Conferir vCPU compartilhada/dedicada e política de CPU sustentada, espaço realmente útil, IOPS/latência, arquitetura, acesso administrativo, virtualização necessária, possibilidade de upgrade, tráfego e preços de renovação/backup. Não selecionar fornecedor só por número nominal de vCPU. Não há preço ou fornecedor aprovado neste kit.

Ensaio e critérios de escala

Registrar build/checks reais, pico de RAM, CPU, I/O, OOM, swap, disco e latência administrativa com job ativo. Definir meta provisória p95 API de 1s nas operações administrativas comuns, sem incluir upload/download/build; validar baseline e instrumentação.

Pausar admission se RAM acima de 85% por 5 min, disco livre abaixo de 20%, OOM ou perda de heartbeat. Se build não cabe no envelope, subir perfil/otimizar antes de diminuir isolamento. Ampliar CPU se saturação sustentada e fila/latência ruins; ampliar RAM se picos/OOM; ampliar disco se retenção/caches excederem envelope. Validar novamente após mudança do piloto.

Segurança e disponibilidade

Usuários/redes/volumes separados, bancos não públicos, credenciais apenas no runtime oficial, root restrito ao gestor confiável, sem socket Docker no código do piloto. Containers compartilham kernel; a VPS única é ponto único de falha. RPO/RTO e restauração precisam de ensaio real. Backup externo pode usar serviço de armazenamento, sem exigir segunda VPS de execução.

ADR-002 - Controle e execução na mesma VPS

Data: 2026-09-29. Decisão de topologia ACEITA pelo usuário; implementação PLANEJADA.

Contexto

A v2 separava VPS de controle e worker no MacBook. O usuário optou por executar também clientes oficiais, autenticação, builds e testes na VPS.

Decisão

VPS Linux única com separação lógica de controle, supervisor, runtime de agentes e código do piloto. Assinaturas via clientes oficiais; APIs/extras desligados. Perfil Bom (8 vCPU/16 GB/200 GB) como recomendação de dimensionamento, contratação ainda pendente.

Consequências

Independência do MacBook e operação centralizada. Maior VPS, sessões pessoais no servidor, disputa de recursos e ponto único de falha. Isolamento precisa impedir acesso do código às credenciais e ao controle. Revisão e merge/deploy mantêm políticas anteriores.

Verificação antes de ativar

Login oficial headless e em identidade de serviço; termos/cobrança compatíveis; teste de segredo inacessível; build/testes dentro do envelope; cancelamento/lease, reboot e restauração. Não houve implantação nesta revisão.

Evolução

Preservar contratos de worker para futura separação se risco, carga ou disponibilidade justificarem. Não exigir segundo servidor no MVP.

Plano do MVP v2

Um módulo existente, um projeto, um worker, um writer. Sprints por objetivos de 1-2 semanas sugeridas, sem datas contratuais. Somatório estimado: 23-35 dias de engenharia para FAC-001 a FAC-012; calendário depende de disponibilidade e compatibilidade. FAC-013 é evolução opcional.

V0 assistida entrega aprendizado antes da plataforma; V1 valida runtime único; V2 testa handoff; Sprint 3 mede operação. Nenhum ticket está implementado.

Épicos

Sprint 0: contrato e viabilidade. Sprint 1: controle, worker e runtime. Sprint 2: execução recuperável e providers. Sprint 3: documentação, painel e experimento. Evolução: terceiro adapter/capacidades novas.

FAC-001 — Contratar piloto

Sprint: Sprint 0. Dependências: nenhuma. Esforço estimado: 1-2 dias. Status: PLANEJADO.

Descrição: Inspeccionar repo, baseline, regras, caminhos e três tickets pequenos.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Contrato verificável e baseline reproduzido; não editar lógica fora do ticket.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-002 — Validar clientes e baseline assistido

Sprint: Sprint 0. Dependências: FAC-001. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Preflight de login/cobrança/isolamento na VPS selecionada; medir baseline de recursos, executar três tickets e handoff manual; confirmar providers um a um.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Um adapter elegível comprovado, três entregas e checkpoint recuperável; limites desconhecidos explícitos.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-003 — Controle web e persistência

Sprint: Sprint 1. Dependências: FAC-002. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Selecionar e registrar perfil mínimo/bom/ideal, provisionar VPS única, redes/volumes/limites e backup externo; bootstrap proposto, auth administrativa, entidades/tickets/attempts e outbox; Redis/Postgres privados.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Ticket autenticado persiste após reinício e dispatch idempotente; sem segredo em Git.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-004 — Worker interno e identidade de serviço

Sprint: Sprint 1. Dependências: FAC-003. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Registro restrito, claim, heartbeat, eventos, systemd, login oficial na identidade de serviço e limites globais; rede interna autenticada.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Queda de rede interrompe writer antes do lease; job antigo não recebe conclusão válida; sem portas públicas de worker/banco/fila.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-005 — Runtime Gateway e primeiro adapter

Sprint: Sprint 1. Dependências: FAC-002, FAC-004. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Interface, execução segura por argv/stdin, fixtures/eventos e cliente oficial escolhido por preflight.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Execute/cancel/schema comprovados; auth/cota normalizados; modelo/uso desconhecido não inventados.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-006 — Context Builder e RuntimeGuard

Sprint: Sprint 1. Dependências: FAC-001, FAC-003. Esforço estimado: 1-2 dias. Status: PLANEJADO.

Descrição: Fontes determinísticas/hashes; limites de tentativas/tempo/troca; API/extra bloqueados.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Sem segredo/dependência no contexto; falha repetida pausa; keys herdadas não ativam API.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-007 — Sandbox e snapshots recuperáveis

Sprint: Sprint 2. Dependências: FAC-004, FAC-005. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Worktree isolado, recursos, serviços sintéticos, patches/untracked e revisão exata.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Sem host socket/home/segre-dos; timeout mata árvore; snapshot restaura arquivos rastreados e untracked.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-008 — Orquestrador e checkpoints

Sprint: Sprint 2. Dependências: FAC-006, FAC-007. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Estados/outbox/leases, writer lock, pausa/cancel/resume e recovery conservador.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Lease expirado sem quiescência bloqueia novo writer; eventos duplicados não repetem efeitos; retorno seguro após crash.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-009 — Developer, checks e revisão

Sprint: Sprint 2. Dependências: FAC-008. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Executar ticket, checks do repo, reviewer separado e correções limitadas.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Critérios comprovados na revisão exata; falhas anteriores separadas; até duas correções sem loop.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-010 — Segundo provider e handoff automático

Sprint: Sprint 2. Dependências: FAC-005, FAC-008, FAC-009. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Preflight segundo adapter, roteamento por capacidade e troca após término confirmado.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Simular cota, preservar patch/untracked, outro cliente continua sem writer concorrente; sem provider aguarda.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-011 — Gate documental e Provider Manager

Sprint: Sprint 3. Dependências: FAC-009, FAC-010. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Docs/lessons/índices, painel móvel, observações de cota com fonte e aceite por revisão.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README

atual, lessons pertinentes e backlog.

Aceite: Mudança sem docs falha; unknown visível; aprovação obsoleta rejeitada; docs atuais coerentes com código.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-012 — Dez tickets e operação

Sprint: Sprint 3. Dependências: FAC-011. Esforço estimado: 2-3 dias. Status: PLANEJADO.

Descrição: Ensaio incluindo falhas, backup/restauração, métricas de custo/espera/qualidade e retrospectiva.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Dez tickets contabilizados; restauração comprovada; metas avaliadas; zero cobrança extra/API não autorizada.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

FAC-013 — Terceiro adapter e QA UI

Sprint: Evolução. Dependências: FAC-012. Esforço estimado: 1-3 dias. Status: PLANEJADO.

Descrição: Validar provider restante e browser efêmero se piloto precisar.

Subtarefas: registrar contrato e evidência de início; implementar menor incremento; verificar cenários de sucesso/falha descritos; revisar diff/limites; atualizar relatório por domínio, README atual, lessons pertinentes e backlog.

Aceite: Capacidades/termos/login/uso comprovados; critérios UI com evidências; sem sessão pessoal.

Entregáveis: código quando pertinente, checks reais, artefatos por revisão e documentação.

Definition of Done

Aceite/critério com evidência, checks da revisão final, revisão resolvida, diff recuperável, estado de uso/custo honesto, docs atuais e relato datado, lessons pertinentes, índice/backlog atualizados e aceite humano. Sem merge/deploy automático.

Experimento

Registrar todos os dez tickets, inclusive fracassos. Meta 8/10 em até duas correções; 100% documentados; zero extras/API não autorizados e zero writers concorrentes. Medir duração ativa/espera, retrabalho, handoffs, regressões, custos atribuídos e recursos. Decidir expandir/corrigir/pausar com dados; não adicionar segundo worker ou Temporal por antecipação.

Infraestrutura obrigatória do MVP

Toda execução fica na mesma VPS; MacBook não é dependência. Assinaturas autenticadas nos clientes oficiais da VPS. Estimativas: Mínimo 4 vCPU/8 GB/120 GB; Bom recomendado 8 vCPU/16 GB/200 GB; Ideal 8 vCPU/32 GB/300 GB. Sem GPU. Um executor inicial em todos os perfis.

FAC-002 valida compatibilidade, login, extras desativados e baseline na VPS. FAC-003 registra plano contratado, limites e separação de redes/volumes. FAC-004 testa serviço/credenciais após reboot. FAC-007 prova que código não lê credenciais/controle. FAC-012 mede painel sob build, OOM/I/O/disco e restauração externa.

Aceite de infraestrutura: specs reais documentadas; checks do piloto cabem no envelope; sem OOM no ensaio; bancos privados; segredo inacessível ao código; nenhum efeito duplicado após reinício; backup externo restaurável. Na ocorrência de falha de capacidade, otimizar/subir perfil antes de ampliar concorrência.

Perfil Bom é recomendação de engenharia, não contratação autorizada. Preços, fornecedor e acesso ainda precisam ser informados para provisionar. Detalhamento em `documentacoes/infraestrutura/DIMENSIONAMENTO-VPS.md`.

Especificação implementável v2

Status: PLANEJADO. Interfaces da fábrica a implementar; nenhum contrato abaixo é API de fornecedor.

Dados e invariantes

Project: repo/ref, policies, caminhos e checks cadastrados. Ticket: objetivo/aceite, risco, runtime_limits, budget opcional, status. Run: revisão/base, branch, estado, policy_version e version para lock otimista. Step/Attempt: papel, tentativa, provider, lease_owner/expires, fencing_token, processo/session IDs quando observáveis.

Worker: ID, capacidades verificadas, OS/arch/versão, last heartbeat, disponibilidade e resource limits. ProviderInstallation: worker/provider, cli_version, auth_method, billing_mode, models_verified, capabilities, status e evidência de preflight; sem tokens pessoais.

UsageObservation: instalação, janela/modelo/unidade, valor nullable, reset nullable, fonte e observed_at. AgentExecution: attempt, input/context hashes, result status, modelo efetivo nullable, usage nullable, limites e timestamps. Checkpoint: code SHA, patch/untracked artifacts, stopped_confirmed, next actions e schema_version.

ContextManifest: revisão, fontes/hashes, tamanho estimado, omissões/truncamentos e instruction hashes. Artifact: caminho validado/hash/tamanho/tipo/retention. Approval: actor, run e revisão exata. LedgerEntry: fixed_subscription/extra/api, moeda, valor decimal nullable, alocação/estimativa/reconciliação e fonte.

Constraints: event_id e sequence únicos por attempt; um writer por workspace; transições versionadas; aprovação vinculada ao code SHA; dispatch via outbox; artefatos imutáveis por hash. Nunca dinheiro em float.

APIs do controle

POST /projects; POST /tickets; POST /tickets/{id}/runs com Idempotency-Key; GET /runs/{id}/events; POST /runs/{id}/pause, cancel, resume, approvals. GET /workers; GET /providers; PATCH /providers/{id}/policy. Provider policy não aceita senha/token OAuth nem pode habilitar gasto via evento de agente.

Worker protocol em documentacoes/operacao/README.md. Autorização por projeto/worker, limites de upload, sanitização e audit log. 409 para estado/revisão obsoleta; 422 contrato incompleto; 401/403 identidade/permissão.

Máquina de estados

DRAFT -> READY -> WAITING_WORKER -> RUNNING -> VALIDATING -> REVIEW -> DOCS -> AWAITING_HUMAN -> DONE. Estados laterais WAITING_PROVIDER, PAUSED_LIMIT, PAUSED_RESOURCE, BLOCKED_RECOVERY, AUTH_REQUIRED, FAILED e CANCELLED. Resume é nova tentativa baseada em checkpoint persistido; não salto direto a DONE.

Cada transição guarda ator/evento/revisão/motivo. DONE requer aceite; cancelamento pode terminar como resultado desconhecido até confirmar fim. Uso indisponível não pode impedir relatório honesto, mas precisa constar desconhecido.

Fluxo

Inspeção determinística -> contexto -> lock writer -> adapter execute -> checks reais -> reviewer separado -> correção limitada -> documentação -> gate -> aceite. Cliente oficial executa tools dentro do isolamento; supervisor controla lifecycle e valida outputs. Não tratar evento de IA como comando confiável.

Gate documental

Comparar base/code revision, exigir relato por domínio, diff/patch associado, checks com resultados e revisão, README atual e atualização de índice/changelog/backlog. Validar links/estrutura, evitar placeholder em relatório IMPLEMENTADO e segredos. Revisão semântica confere conteúdo; presença de arquivo não prova verdade.

Checks essenciais do MVP

Idempotência de claim e eventos; worker desconectado interrompe writer; processos antigos não continuam após handoff; arquivos untracked recuperáveis; auth/cota tratados sem loop; API key herdada bloqueada; extras desligados verificados; segredo inacessível ao código do piloto; doc gate rejeita entrega incompleta; aceite invalidado por novo diff.

Mocks/fixtures sanitizadas para adapters; integração real opt-in no preflight dentro da assinatura validada. Medir ensaio de falha por simulação de adapter, sem desperdiçar cota real de propósito.

Agent Runtime Gateway e Provider Manager

Status: contrato PLANEJADO, nenhum adapter testado.

Adapters

Codex: `codex exec --json --sandbox workspace-write`, prompt via stdin ou argumento seguro; `--output-last-message` quando suportado. Claude: `claude -p --output-format json`, permissões mínimas validadas. Antigravity: `agy -p`; validar flags de saída/permissões pela versão instalada antes de assumir JSON.

Exemplos acima descrevem invocação, não autorizam executar código com privilégios. Usar `spawn/execFile` com array de argumentos e stdin, jamais interpolar prompt em shell. Fixar versão/binário; validar origem oficial e registrar checksums quando disponíveis.

Clientes CLI executam ferramentas autonomamente. Adaptador coleta eventos e controla processo; o host impõe recursos, mounts, rede e identidade. Não fingir que `tool_requests` da API continua idêntico ao loop interno do CLI.

Preflight por instalação

Registrar OS/CPU, versão, binário, método oficial de login e plano declarado, modelos realmente acessíveis, modo não interativo, leitura das regras, sandbox, permissão de ferramentas, formato de eventos, cancelamento e comportamento de limite. Desabilitar API keys herdadas e verificar extras desligados. Rodar um ticket sintético de leitura e depois alteração pequena com checks.

Testar credenciais no serviço real: login num terminal não prova acesso pelo usuário `systemd/container`. Usar armazenamento oficial suportado; não copiar sessões entre máquinas como atalho. Se containerização não suporta login seguro, manter adapter bloqueado até ADR de isolamento compatível.

Cada capability carrega `verified_at`, `cli_version` e `evidence_id`. Recursos desconhecidos são `unsupported`; `getUsage` pode retornar `UNKNOWN`. Um provider não passa a `AVAILABLE` apenas porque existe no catálogo.

Interface proposta

`execute(request)` -> event stream + result; `getStatus()` -> health/auth/limits; `getCapabilities()` -> capacidades verificadas; `getUsage()` -> observações nullable; `cancel(execution_id)` -> estado e confirmação de processos parados; `resume(checkpoint)` -> nova tentativa ou `unsupported`.

Request inclui `run_id`, `attempt_id`, `fencing_token`, `provider_installation_id`, `model_requested` nullable, `workspace_id`, `base_sha/code_sha`, `context_manifest`, `instruction_hashes`, `command_profile`, limites e `policy_version`.

Resultado inclui `schema_version`, `status`, `exit_code`, `model_effective` nullable, `provider_session_id` nullable, revisão, patch/artifact hashes, checks, summary, usage nullable, billing_mode, timestamps e erro normalizado. Não capturar raciocínio interno privado.

Eventos: `started`, `progress`, `checkpoint`, `usage_observed`, `limit_observed`, `artifact_ready`, `process_exited` e `finished`. A fábrica valida sequência/schema; output do agente é dado não confiável e não pode alterar orçamento/política.

Erros normalizados

AUTH_REQUIRED: usuário autentica no fluxo oficial. RATE_LIMITED: cooldown/fallback elegível. TRANSIENT: backoff com limite. CONTEXT_TOO_LARGE: reduzir contexto e registrar mudança. TOOL_DENIED: corrigir perfil/escopo, não ampliar automaticamente. TIMEOUT: cancelar árvore, checkpoint e revisão de estado. RESULT_UNKNOWN: reconciliar antes de repetir. UNSUPPORTED: manter adapter desativado para a capacidade.

Uso e cotas têm unidade/fonte/observed_at/reset_at/confiança. Estado de quota, crédito de assinatura e tarifa API são dados distintos. Não usar endpoints privados ou scraping do site para medir cota.

Roteamento

Filtrar por política/capacidade e disponibilidade; ordenar por preferência do papel e desempenho recente; adquirir lock do writer; executar. Preferências iniciais configuráveis: implementação Codex; revisão/arquitetura Claude; UI/QA Antigravity. Não usar percentuais fictícios de sucesso; aprender com ensaio registrado.

Cache de roteamento não deve ignorar observações novas de limite. Provider indisponível não implica mudar modelo/conta silenciosamente. API só entra no catálogo quando habilitada explicitamente em política futura.

Checkpoint e troca de provider

Regra central

Um writer ativo por workspace. Estado externo é a memória compartilhada: PostgreSQL para workflow, Git para código/docs, armazenamento de artefatos para patch/untracked/checks e checkpoint para próximos passos. Não transferir sessão privada de um fornecedor a outro.

Procedimento

1. Solicitar pausa e bloquear novas ferramentas/etapas. Se cliente não pode pausar, cancelar árvore e aguardar término comprovado.
2. Supervisor coleta status Git, base/code SHA, diff binário quando aplicável e lista de untracked; exclui segredos e dependências. Fazer snapshot verificável mesmo que o agente não consiga gerar resumo após esgotar cota.
3. Persistir artefatos com hashes e checkpoint. Commit WIP só se permitido no repo e conteúdo revisado; alternativamente guardar patch + untracked. Commit sozinho não cobre arquivos não rastreados.
4. Confirmar envio/recuperabilidade, liberar lock anterior e emitir novo fencing_token. Novo job rejeita eventos do token antigo.
5. Novo provider lê checkpoint, confere revisão e patch, reexecuta checks pertinentes e continua somente próximos passos. Não reaplicar patch já presente.
6. Registrar handoff, motivo, origem/destino, duração e resultado. Limite proposto de dois handoffs; depois pausa com diagnóstico.

Corrida de recuperação

Lease expirado na VPS não prova que processo local morreu. Durante partição de rede, o worker deve se auto-interromper antes de vencer sua janela de lease. Controle não envia outro writer ao mesmo workspace até confirmação de quiescência; sem confirmação, manter BLOCKED_RECOVERY. Não considerar fencing_token suficiente para impedir dois processos editando arquivos locais.

Heartbeat proposto 15s, lease 90s; worker precisa parar antes de expirar, incluindo margem de encerramento. Valores devem ser ensaiados. Supervisor sem lease não inicia comandos. Cancelamento registra resultado desconhecido caso não possa confirmar término.

Conteúdo mínimo

Ticket/run/attempt, objetivo/aceite, escopo, base SHA/code SHA, branch/workspace, provider/versão/modelo conhecido, arquivos modificados/untracked, patch hash, decisões e evidências, testes reais e revisão, falhas/limitações, próximos passos, docs pendentes, motivo da pausa e autenticação/cota observada sem segredos.

Checkpoint não afirma aceite nem transforma checks antigos em atuais. Cancelamento pode ter consumido cota; guardar uso desconhecido como desconhecido.

Operação da fábrica e worker

Inicialização

Confirmar piloto e políticas; instalar clientes de fontes oficiais com versões registradas; login humano nos clientes; verificar extras desligados nas contas; executar preflight; registrar worker com credencial própria de escopo mínimo e validade/rotação. Deploy do controle é uma tarefa futura autorizada separadamente.

VPS roda proxy HTTPS, frontend, API, scheduler, PostgreSQL e Redis privados. Supervisor interno roda como usuário dedicado e usa protocolo interno autenticado. Manter credencial da fábrica separada de credenciais dos providers. Não expor endpoint local ou montar diretórios pessoais nos worktrees.

Sandbox sem privileged, socket Docker, home completo, banco/Redis da fábrica, credenciais de Git amplas ou dados reais. Supervisor confiável prepara checkout e serviços sintéticos. Credenciais de provider inevitavelmente acessíveis ao cliente exigem isolamento de identidade/armazenamento; código executado não deve conseguir lê-las. Validar por teste de acesso negado, não só instrução escrita.

Worker API

POST /workers/register (onboarding restrito), POST /workers/{id}/heartbeat, POST /workers/{id}/jobs/claim, POST /attempts/{id}/events, POST /attempts/{id}/checkpoint, POST /attempts/{id}/complete. Payloads versionados; autenticação por worker; claim transacional; sequence/event_id únicos; fencing_token obrigatório. Artefatos com tamanho/hash/paths permitidos, nunca path traversal.

Worker não escolhe arbitrariamente repo, comando ou URL fornecidos em output de IA. Controle só distribui jobs de projetos cadastrados; valida repo/ref e perfis de comando. Eventos/logs são sanitizados e limitados antes de armazenar/exibir.

Recuperação

Queda do worker: parar agendamento local, manter painel, esperar heartbeat; preservar checkpoints. Internet caiu: parar execução antes de lease expirar. Auth expirada: AUTH_REQUIRED; usuário faz login oficial local. Cota acabou: handoff seguro ou WAITING_PROVIDER. Disco cheio/OOM: PAUSED_RESOURCE, limpar somente workspaces já preservados. CLI mudou schema: bloquear adapter, testar fixture e integrar atualização em ticket próprio.

Kill switch bloqueia claims, solicita cancelamento e monitora fim das árvores de processos; nunca declarar cancelado sem evidência. Supervisor aplica timeout e limites de logs/disco, além de CPU/RAM.

Backup e restauração

Restaurar PostgreSQL numa instância de teste; conferir tickets e versões; recuperar artefatos por hash; buscar Git/revisão; renovar credencial do worker sem restaurar tokens de provider por cópia informal. Testar cenário com execução interrompida. Registrar tempo real/RPO efetivo e lacunas.

Observabilidade

Fila/etapa, last heartbeat, lease, processos, CPU/RAM/disco, OOM, duração, provider/modelo/versão, uso conhecido/desconhecido, handoffs, retries, checks/docs e revisão aceita. Evitar prompts completos e segredos nos logs. Browser de QA com perfil efêmero sem sessão pessoal.

Evolução

Separar execução em outra VPS é alternativa futura, sujeita a métricas e revisão de isolamento. APIs podem prover capacidade com orçamento; execução na VPS não elimina termos dos fornecedores. Código de terceiros exige isolamento mais forte que containers compartilhando host.

Provisionamento da VPS única no MVP

Selecionar perfil em infraestrutura/DIMENSIONAMENTO-VPS.md e registrar especificações reais. Configurar firewall: HTTPS público; SSH administrativo restrito; Redis/PostgreSQL/worker privados. Separar volumes persistentes de temporários. Instalar supervisor como serviço com usuário dedicado e recuperação após reinício.

Fazer login oficial pelo método suportado para servidor sem interface; callback/fluxo remoto apenas conforme documentação do cliente. Não desabilitar autenticação nem copiar cookies. Confirmar credenciais acessíveis ao cliente e inacessíveis a código/testes/logs.

Rodar baseline no mesmo ambiente que executará jobs. Medir recursos e painel durante build. Reiniciar worker, depois VPS em janela de teste; comprovar leases, nenhuma duplicação e retomada de checkpoint. Validar backup externo e restauração. Não abrir banco/fila para facilitar diagnóstico.

Política de economia

A prioridade é economizar capacidade e evitar retrabalho. Assinaturas substituem cobrança por chamada na arquitetura principal, dentro da elegibilidade e limites reais de cada plano.

1. Ticket pequeno, critérios claros e baseline antes da IA.
2. rg/imports/testes para selecionar fontes; excluir vendor/node_modules/dumps/segredos.
3. Apenas Developer e Reviewer no fluxo comum; QA automatizado e documentação integrada. Papéis adicionais por necessidade.
4. Sessão independente para review, contexto resumido mas com fontes acessíveis.
5. Limitar correções, timeout, handoffs e concorrência; pausar se diagnóstico repetido.
6. Escolher modelo exposto/elegível adequado e medir qualidade. Contextos curtos ajudam a cota, mas limites não são conversão fixa tokens/tickets.
7. Reusar caches de dependências e outputs determinísticos por revisão. Checks antigos não validam diff novo.
8. Handoff objetivo com artefatos; não colar conversas inteiras entre providers.
9. API, extra usage e créditos automáticos desligados. Aguardar cota se todos indisponíveis.
10. Revisar resultados após dez tickets; custo atribuído, incremental, espera por cota e retrabalho separados.

Não prometer gasto total R\$0: assinaturas, VPS e energia têm custo. Não contratar plano adicional antes de medir gargalo. Preços não foram fixados no kit; preencher com cobrança real e data na configuração financeira.

Política comum de engenharia e documentação

Antes da implementação

Ler README.md, PILOTO.md, PLANO-MVP.md, arquitetura, ADRs e regras locais. Inspecionar o código: documentos não substituem a realidade. Preservar instruções existentes; conflitos relevantes devem ser apresentados ao responsável. Executar apenas um ticket READY com objetivo, escopo, critérios e orçamento definidos.

Implementação

Preservar stack e padrões do repo alvo. Não ampliar escopo, migrar linguagem ou adicionar dependência sem necessidade. Planejar curto, trabalhar em branch isolada, limitar ferramentas e nunca acessar produção. Não inserir segredos em prompts, arquivos ou logs. Clientes oficiais autenticados pelo usuário; API e extras desligados no MVP. Escolha somente provider elegível no catálogo validado.

Rodar os checks pertinentes e relatar comandos/resultado real. Distinguir falha de baseline de regressão. Não simular sucesso. Revisão recebe diff, critérios e evidências. Corrigir no máximo duas rodadas antes de pausar com diagnóstico.

Entrega obrigatória em toda alteração

1. Criar documentacoes/<dominio>/AAAA-MM-DD-TICKET-titulo.md, usando o template.
2. Atualizar documentacoes/<dominio>/README.md para descrever estado atual e funcionamento, contratos e exemplos reais.
3. Atualizar arquitetura e criar ADR se mudou decisão estrutural; revisar API, configuração e operação afetadas.
4. Atualizar documentacoes/INDEX.md, CHANGELOG.md e backlog. Não marcar DONE sem aceite.
5. Criar/atualizar lessons/<conceito>.md apenas quando conceito foi aplicado; atualizar índice, evitando duplicação.
6. Associar evidência à revisão exata. Incluir diff relevante sanitizado ou caminho de patch revisável. Jamais colar segredo, dump ou dados pessoais.
7. Encerrar com o que mudou, como funciona, verificações, limites, rollback e documentação atualizada.

Histórico de entregas é append-only salvo correção identificada; docs atuais são atualizadas em toda mudança pertinente. Comentários de código não substituem relatório.

Regras de veracidade

Usar IMPLEMENTADO, PLANEJADO ou NÃO VERIFICADO. Não escrever que foi executado algo apenas recomendado. Datas reais no fuso do responsável; IDs reais; hashes reais. Evitar hash circular: usar revisão do código anterior ao commit documental e/ou link do PR final.

Bloqueios

Ausência de repo/escopo/critério impede a implementação do piloto, mas permite inspeção e planejamento. Budget esgotado impede novas chamadas. Merge/deploy, ações destrutivas e migrações irreversíveis dependem de autorização explícita.

Execução e handoff

Ler o checkpoint e conferir SHA, diff e arquivos não rastreados antes de retomar. Nunca depender da memória da conversa anterior. Um writer por worktree; não iniciar outro até confirmar término da árvore de processos anterior. Se não houver provider, preservar trabalho e aguardar. Não alterar login, plano, créditos ou política financeira para continuar.

Relatar provider, versão, modelo efetivo quando conhecido, tempo, uso reportado, fonte e dados desconhecidos. Não converter tokens de assinatura automaticamente em cobrança. Não fingir que observação antiga de cota é atual.

Documentar também entregas parciais/interrompidas e próximos passos; checkpoints não substituem relatório final. Descrever PLANEJADO quando ainda não implementado.

Contexto e verificação

Não carregar todo o kit a cada ticket. Ler política comum e documentos relevantes ao domínio; Context Builder guarda hashes e omissões. Executar checks na revisão final e invalidar evidência após alteração pertinente. Gate de documentação estrutural precisa de revisão semântica.

Topologia obrigatória v2.1

Controle, worker, clientes oficiais/autenticação e sandbox executam na mesma VPS. Seguir ADR-002 e infraestrutura/DIMENSIONAMENTO-VPS.md (sob documentacoes). Não depender de MacBook. Um executor inicial; impor limites globais e preservar controle/banco/credenciais fora do alcance do código. Não contratar VPS ou habilitar gastos sem autorização aplicável.

Guia para Codex e agentes do repositório

Leia documentacoes/POLITICA-IA.md integralmente, README.md, PILOTO.md, o ticket e os documentos dos domínios afetados. Respeite regras locais mais específicas e preserve arquivos existentes ao integrar este guia.

Trabalhe em um ticket READY e uma branch/worktree isolada. O objetivo da fábrica é VPS única de controle e execução + clientes oficiais com assinaturas. Não reintroduza API como padrão. Não habilite extra usage, créditos, autorecharge ou fallback pago.

Antes de editar: confirme objetivo, caminhos, critérios, baseline, provider elegível e limites. Se houver handoff, confira SHA, patch, untracked e evidências. Não iniciar execução se outro writer não estiver comprovadamente parado.

Implemente incremento pequeno; execute checks adequados; diferencie regressão de falha anterior. Até duas rodadas de correção; depois checkpoint e diagnóstico. Não declarar sucesso só por exit code ou mensagem do modelo.

Toda feature/correção/configuração/refatoração exige documentacoes/<dominio>/AAAA-MM-DD-TICKET-titulo.md, README atual do domínio, índice e changelog/backlog atualizados, diff sanitizado e evidências reais. Atualizar lessons com conceitos realmente aplicados e exemplos do repo, evitando duplicação. Atualizar ADRs/contratos/operação quando afetados. Nunca finalizar com documentação pendente.

Final: o que mudou, funcionamento, checks/resultados, limitações, rollback, docs/lessons e estado do aceite. DONE só após aceite da revisão exata. Não realizar merge/deploy ou ação destrutiva sem autorização aplicável.

Codex

AGENTS.md é o ponto de entrada do Codex; confirmar hierarquia de instruções na versão instalada. Codex pode atuar como implementador ou revisor; papel é definido pelo ticket, não pela marca.

Adapter inicial usa codex exec com eventos JSONL, sandbox explícito e autenticação salva do cliente oficial na VPS. Validar --help e versão; não usar permissões amplas como padrão. Modelo efetivo vem de evidência do cliente, não de suposição.

Resume nativo só quando sessão/ambiente compatíveis; handoff para outro provider sempre usa estado externo. Não exportar auth.json ou tokens ao controle. Se API key estiver herdada, impedir uso no modo subscription-only.

Topologia obrigatória v2.1

Controle, worker, clientes oficiais/authenticação e sandbox executam na mesma VPS. Seguir ADR-002 e infraestrutura/DIMENSIONAMENTO-VPS.md (sob documentacoes). Não depender de MacBook. Um executor inicial; impor limites globais e preservar controle/banco/credenciais fora do alcance do código. Não contratar VPS ou habilitar gastos sem autorização aplicável.

Guia para Claude Code

Leia documentacoes/POLITICA-IA.md integralmente, README.md, PILOTO.md, o ticket e os documentos dos domínios afetados. Respeite regras locais mais específicas e preserve arquivos existentes ao integrar este guia.

Trabalhe em um ticket READY e uma branch/worktree isolada. O objetivo da fábrica é VPS única de controle e execução + clientes oficiais com assinaturas. Não reintroduza API como padrão. Não habilite extra usage, créditos, autorecharge ou fallback pago.

Antes de editar: confirme objetivo, caminhos, critérios, baseline, provider elegível e limites. Se houver handoff, confira SHA, patch, untracked e evidências. Não iniciar execução se outro writer não estiver comprovadamente parado.

Implemente incremento pequeno; execute checks adequados; diferencie regressão de falha anterior. Até duas rodadas de correção; depois checkpoint e diagnóstico. Não declarar sucesso só por exit code ou mensagem do modelo.

Toda feature/correção/configuração/refatoração exige documentacoes/<dominio>/AAAA-MM-DD-TICKET-titulo.md, README atual do domínio, índice e changelog/backlog atualizados, diff sanitizado e evidências reais. Atualizar lessons com conceitos realmente aplicados e exemplos do repo, evitando duplicação. Atualizar ADRs/contratos/operação quando afetados. Nunca finalizar com documentação pendente.

Final: o que mudou, funcionamento, checks/resultados, limitações, rollback, docs/lessons e estado do aceite. DONE só após aceite da revisão exata. Não realizar merge/deploy ou ação destrutiva sem autorização aplicável.

Claude

Use cliente oficial sem modificar fluxo de autenticação. claude -p é o caminho não interativo; validar modo de cobrança/plano antes de ativá-lo. Não usar token OAuth do Claude Code como credencial de SDK ou chamada HTTP própria.

Este guia é carregado quando suportado pela versão/mode. --bare ignora CLAUDE.md e várias descobertas: se adotado, a fábrica precisa injetar política explicitamente, registrar hash e justificar a escolha. Não assumir regras carregadas.

Para review, usar sessão separada sem edição da revisão analisada e fornecer achados concretos com caminhos/critério. Preferência inicial por arquitetura/revisão é configurável e precisa de métricas. Não adicionar agentes paralelos automaticamente, pois consomem cota.

Topologia obrigatória v2.1

Controle, worker, clientes oficiais/authenticação e sandbox executam na mesma VPS. Seguir ADR-002 e infraestrutura/DIMENSIONAMENTO-VPS.md (sob documentacoes). Não depender de MacBook. Um executor inicial; impor limites globais e preservar controle/banco/credenciais fora do alcance do código. Não contratar VPS ou habilitar gastos sem autorização aplicável.

Guia para Antigravity

Leia documentacoes/POLITICA-IA.md integralmente, README.md, PILOTO.md, o ticket e os documentos dos domínios afetados. Respeite regras locais mais específicas e preserve arquivos existentes ao integrar este guia.

Trabalhe em um ticket READY e uma branch/worktree isolada. O objetivo da fábrica é VPS única de controle e execução + clientes oficiais com assinaturas. Não reintroduza API como padrão. Não habilite extra usage, créditos, autorecharge ou fallback pago.

Antes de editar: confirme objetivo, caminhos, critérios, baseline, provider elegível e limites. Se houver handoff, confira SHA, patch, untracked e evidências. Não iniciar execução se outro writer não estiver comprovadamente parado.

Implemente incremento pequeno; execute checks adequados; diferencie regressão de falha anterior. Até duas rodadas de correção; depois checkpoint e diagnóstico. Não declarar sucesso só por exit code ou mensagem do modelo.

Toda feature/correção/configuração/refatoração exige documentacoes/<dominio>/AAAA-MM-DD-TICKET-titulo.md, README atual do domínio, índice e changelog/backlog atualizados, diff sanitizado e evidências reais. Atualizar lessons com conceitos realmente aplicados e exemplos do repo, evitando duplicação. Atualizar ADRs/contratos/operação quando afetados. Nunca finalizar com documentação pendente.

Final: o que mudou, funcionamento, checks/resultados, limitações, rollback, docs/lessons e estado do aceite. DONE só após aceite da revisão exata. Não realizar merge/deploy ou ação destrutiva sem autorização aplicável.

Carregamento e uso

ANTIGRAVITY.md é guia do projeto; não presumir que seja nome reservado. A regra comum em .agents/rules/documentacao.md também precisa de teste de carregamento na versão/superfície instalada. Se ignorada, incluir guia e política explicitamente no prompt do job.

Google documenta agy -p para automação. Confirmar versão, modelos, autenticação, saída, permissões e créditos no preflight; browser/E2E só quando capacidade comprovada. Não assumir que todo modelo listado é coberto pelo plano.

UI/QA deve testar critérios com dados sintéticos e anexar evidências da revisão exata. Capturas não provam todas as regras de negócio; checks determinísticos continuam necessários. Navegador sem sessões pessoais, contas de produção ou credenciais reais. Modo always-proceed não substitui isolamento nem é padrão do MVP.

Topologia obrigatória v2.1

Controle, worker, clientes oficiais/authenticação e sandbox executam na mesma VPS. Seguir ADR-002 e infraestrutura/DIMENSIONAMENTO-VPS.md (sob documentacoes). Não depender de MacBook. Um executor inicial; impor limites globais e preservar controle/banco/credenciais fora do alcance do código. Não contratar VPS ou habilitar gastos sem autorização aplicável.

Contrato do piloto

Status: PLANEJADO — completar antes de execução.

- Repositório/acesso, base branch/SHA: A DEFINIR.
- Funcionalidade pequena e critérios verificáveis: A DEFINIR.
- Stack/versões/checks/baseline: inspecionar repo real.
- Caminhos permitidos/proibidos: A DEFINIR.
- Infraestrutura: VPS única; perfil Bom recomendado 8 vCPU/16 GB/200 GB; perfil contratado A DEFINIR. Clientes oficiais autenticados na VPS; compatibilidade A VERIFICAR.
- Providers/planos/modos de autenticação/modelos: registrar no preflight real.
- Limites propostos: um writer, 30 min/tentativa, duas correções, dois handoffs.
- API: desativada; orçamento mensal zero. Extra usage/créditos/autorecharge: desativados nos fornecedores.
- Dados somente sintéticos; sem merge/deploy automático.

Preferir filtro, validação ou exibição num módulo existente com testes. Excluir autenticação/pagamento/migração irreversível na primeira amostra. Não assumir qual sistema será usado; pode ser escolhido pelo responsável.

Backlog v2

Nenhum ticket DONE; FAC-001 aguarda definição do repo/piloto. Não preencher prazo contratual sem capacidade definida.

- FAC-001: PLANEJADO — Contratar piloto; dependências: nenhuma.
- FAC-002: PLANEJADO — Validar clientes e baseline assistido; dependências: FAC-001.
- FAC-003: PLANEJADO — Controle web e persistência; dependências: FAC-002.
- FAC-004: PLANEJADO — Worker interno e identidade de serviço; dependências: FAC-003.
- FAC-005: PLANEJADO — Runtime Gateway e primeiro adapter; dependências: FAC-002, FAC-004.
- FAC-006: PLANEJADO — Context Builder e RuntimeGuard; dependências: FAC-001, FAC-003.
- FAC-007: PLANEJADO — Sandbox e snapshots recuperáveis; dependências: FAC-004, FAC-005.
- FAC-008: PLANEJADO — Orquestrador e checkpoints; dependências: FAC-006, FAC-007.
- FAC-009: PLANEJADO — Developer, checks e revisão; dependências: FAC-008.
- FAC-010: PLANEJADO — Segundo provider e handoff automático; dependências: FAC-005, FAC-008, FAC-009.
- FAC-011: PLANEJADO — Gate documental e Provider Manager; dependências: FAC-009, FAC-010.
- FAC-012: PLANEJADO — Dez tickets e operação; dependências: FAC-011.
- FAC-013: PLANEJADO — Terceiro adapter e QA UI; dependências: FAC-012.

Infraestrutura VPS única incluída em FAC-002/003/004/007/012; perfil Bom recomendado.

Cobertura e migração v1 -> v2

Fonte: kit v1 e PDF original recuperados nesta tarefa, mais nova direção fornecida pelo usuário. Estrutura e requisitos anteriores preservados/adaptados; alterações incompatíveis substituídas.

Tema anterior	Tratamento v2	Documento
VPS única para tudo	VPS única para controle e execução	Arquitetura 1-3
IA por API	Clientes oficiais/assinatura; API futura desligada	Runtime; Fontes
AI Gateway	Agent Runtime Gateway, lifecycle real dos CLIs	ESPEC-MVP; Runtime
ModelRouter	Elegibilidade/capacidade/cota e desempenho medido	Arquitetura 7
TokenMeter/CostLedger/BudgetGuard	UsageMeter + custos fixos/extras + RuntimeGuard; API budget futuro	Economia; Especificação
Simulação API por tokens	Substituída por TCO/atribuição; sem promessa de tickets	Arquitetura 8
Context Builder/cache/Batch	Contexto enxuto; cache condicionado; Batch só futuro API	Arquitetura 6; Economia
MacBook fora V1	Sem dependência no MVP; execução na VPS	Operação
Docker/worktrees/testes	Mantidos no worker, isolamento do cliente e código	Arquitetura 9; Operação
Tickets/estados/recuperação	Acrescenta espera, leases, cancel e checkpoint	Especificação; Handoff
Arquiteto/Tech Lead/Developer/Review/QA/docs	Papéis mantidos; mínimo Developer/Reviewer	Arquitetura 5
R0-R4 e escopo pequeno	Mantidos; piloto R0/R1	Arquitetura 5; PILOTO
APIs internas/entidades	Atualizadas com workers/installations/checkpoints	ESPEC-MVP
Documentação por domínio/diffs	Mantida obrigatória em toda entrega	Política; templates
Lessons com exemplos	Mantidas, reais e sem duplicação	Política; template
Guias Claude/Codex/Antigravity	Atualizados para clientes/assinaturas	Três MDs
Sprints/épicos/subtarefas	Replanejados, 12 tickets MVP + 1 evolução	PLANO-MVP; BACKLOG
Backups/disco/retention	VPS única com recuperação a ensaiar	Operação; Arquitetura 3
Dashboard/custos/aceite	Uso observado/unknown, mobile, revisão exata	Arquitetura 12
Temporal/autoscaling/GPU	Evoluções condicionais	Arquitetura 10
Handoff entre providers	Novo contrato seguro incluindo untracked	Handoff; template
Fontes/pendências	Revalidadas; não inventar capacidades	FONTES; PILOTO

Revisão 2.1: VPS única aceita; perfis mínimo/bom/ideal incorporados ao MVP, ADR-002 e dimensionamento. Evidências de implantação seguem pendentes.

Fontes e evidências

Consulta: 29/09/2026. Arquitetura, limites, preferência de papéis e metas são propostas de engenharia. Clientes não foram executados neste trabalho.

OpenAI

- <https://developers.openai.com/codex/noninteractive> — exec, JSONL, sandbox e reutilização de autenticação CLI; endereço redireciona para <https://learn.chatgpt.com/docs/non-interactive-mode>.
- <https://github.com/openai/codex> — cliente oficial local e login ChatGPT. Validar plano/modelos/limites reais no onboarding; não derivar capacidade pelo número de tickets.

Anthropic

- <https://code.claude.com/docs/en/headless> — -p não interativo, saída estruturada e descoberta de regras; --bare ignora CLAUDE.md.
- <https://code.claude.com/docs/en/legal-and-compliance> — autenticação oficial, credenciais e distinção entre uso próprio do binário e serviços para usuários. Não transformar OAuth de assinatura em chave de SDK nem intermediar credenciais.
- <https://code.claude.com/docs/en/costs> — acompanhamento de uso e distinção entre estimativa de tokens e faturamento de assinantes. Condições podem mudar; verificar cobrança do modo automatizado contratado.

Google

- <https://codelabs.developers.google.com/antigravity-cli-hands-on> — instalação/login, agy -p, modelos e controles de permissões/créditos.
- <https://codelabs.developers.google.com/agent-ai-automation-with-antigravity> — CLI e teste de UI; presença de capacidade não prova cobertura de todos os planos/modelos.

Limites da evidência

Não usamos preços fixos ou percentuais de quota. O tutorial técnico não comprova elegibilidade ilimitada, SLA 24/7, quota consultável programaticamente ou reset exato. Essas capacidades precisam de preflight com evidência e versão.

Um resultado de busca sugeriu mudanças de crédito headless do Claude; a página aberta não confirmou essa afirmação. Ela não foi adotada como fato. Validar modo de cobrança real antes de habilitar cada instalação, especialmente SDK/headless.

APIs, caching e Batch do kit anterior continuam extensões possíveis, mas fora da operação MVP. Não somar descontos nem transferir condições de API para assinatura.

Prompt de início

Leia AGENTS.md (Codex), CLAUDE.md (Claude) ou ANTIGRAVITY.md (Antigravity), documentacoes/POLITICA-IA.md, README.md e PILOTO.md. Se o ambiente não carregar regras, considere-as explicitamente parte desta instrução.

Inspecione o estado real do repositório. Comece pelo menor ticket READY; se não houver, execute somente inspeção/planejamento de FAC-001. Não invente repo, modelo, cota ou teste. A arquitetura é assinatura/CLI oficial no worker interno na VPS, controle e execução na mesma VPS, APIs e créditos extras desligados.

Antes de executar, registre contrato, revisão e baseline. Preserve trabalho existente. Entregue código revisável, evidências, relato por domínio, documentação atual e lessons. Em interrupção, checkpoint com patch/untracked e próximos passos. Não implemente todo o backlog numa sessão sem validar incrementos.

ID - título

Status: DRAFT

Objetivo

Atual e esperado

Escopo permitido e proibido

Critérios de aceite verificáveis

Baseline e comandos de checks

Risco e orçamento

Dependências

Entregáveis e documentação afetada

Evidências e aceite

TICKET - título

Data: AAAA-MM-DD

Status: IMPLEMENTADO / PLANEJADO / NÃO VERIFICADO

Domínio:

Base SHA / revisão do código:

Branch / run / PR:

Objetivo e critérios de aceite

O que foi implementado

Arquivos, responsabilidades e decisões reais.

Funcionamento

Entrada, fluxo, regras, saída, erros e exemplos reais.

Alterações e diff

Trecho sanitizado ou caminho do patch associado à revisão do código.

Verificação

Comando; resultado; critério comprovado. Marcar não executado quando aplicável.

Riscos e limitações

Rollback

Passos específicos; se não houver reversão segura, explicar.

Documentação atualizada

Caminhos de README do domínio, ADR/API/operação, índice e backlog.

Lessons

Links ou justificativa de ausência de conceito novo.

Uso de IA e custos

Provider, versão, modo de autenticação/cobrança sem segredo, modelo efetivo, tentativas, handoffs, tempo/tokens observáveis, fonte/horário de quota; custo fixo/extra/API atribuídos e desconhecidos separados.

Pendências e aceite

Responsável e estado; DONE somente após aceite.

HANDOFF — TICKET / RUN

Status: PARCIAL / INTERROMPIDO / PRONTO PARA CONTINUAR

Data e fuso:

Provider/versão/modelo efetivo quando conhecido:

Motivo e limite observado (sem segredo):

Contrato

Objetivo, aceite, escopo e caminhos proibidos.

Estado recuperável

Base/code SHA, branch/workspace, fencing token, confirmação de writer parado, artifact IDs/hashes; untracked e política de exclusão.

O que foi feito

Arquivos, decisões e referências verificáveis.

Checks

Comando, revisão, resultado real, baseline/regressões; não executados.

Próximas ações

Passos prioritários e diagnóstico de falhas.

Documentação

Relato parcial, docs atualizadas/pendentes e lessons.

Uso

Fonte/horário de cota, reset quando informado, tempo/tokens quando observáveis; desconhecido quando ausente.

Conceito

O que é

Por que usamos

Exemplo aplicado

Código/configuração real com caminho e revisão.

Cuidados e alternativas

Evidências e referências

Entregas relacionadas

Documentação atual v2.1

- POLITICA-IA.md (POLITICA-IA.md)
- arquitetura/ADR-002-vps-unica.md (arquitetura/ADR-002-vps-unica.md)
- arquitetura/ARQUITETURA.md (arquitetura/ARQUITETURA.md)
- economia/README.md (economia/README.md)
- handoff/README.md (handoff/README.md)
- infraestrutura/DIMENSIONAMENTO-VPS.md (infraestrutura/DIMENSIONAMENTO-VPS.md)
- operacao/README.md (operacao/README.md)
- runtime/README.md (runtime/README.md)

Lessons

Documentos de conceitos serão preenchidos conforme implementação real. Este kit é planejamento; não inventa exemplos como se executados. Usar templates/LESSON.md e incluir links de revisão/entrega. Conceitos previstos: lifecycle CLI, leases/fencing, worktree/snapshot, contexto determinístico, idempotência, isolamento e cota observada.

Changelog

2.1 - 2026-09-29

Decisão do usuário: toda a fábrica e autenticação na mesma VPS. Dimensionamento mínimo/bom/ideal, limites, ADR-002 e tarefas do MVP atualizados. Guias das três IAs alinhados. Nenhuma implantação realizada.

2.0 — 2026-09-29

Planejamento revisado para assinaturas e clientes oficiais; controle web + worker interno na VPS; gateway/runtime, router/cotas, handoff, isolamento, operação e custos adaptados. Guias das três IAs e templates atualizados. Cobertura v1 rastreada. Nenhum software implementado.

1.0 — 2026-09-29

Kit de planejamento anterior com execução por APIs na VPS, agora substituído nas decisões conflitantes.

Regra comum da fábrica

Ler documentacoes/POLITICA-IA.md e o guia ANTIGRAVITY.md. Aplicar documentação por entrega, atualização de estado atual e lessons. Confirmar carregamento desta regra; não presumir ativação automática.

Políticas de exemplo

```
{
  "schema_version": "2.1",
  "status": "planejado-validar-no-preflight",
  "mode": "personal-subscription-cli",
  "max_concurrent_executors": 1,
  "max_correction_rounds": 2,
  "max_escalations": 1,
  "max_handoffs_per_ticket": 2,
  "monthly_api_budget": 0,
  "fallback_api": {
    "enabled": false
  },
  "paid_extras": {
    "allowed": false,
    "verify_in_provider_account": true
  },
  "lease": {
    "heartbeat_seconds": 15,
    "duration_seconds": 90,
    "stop_before_expiry": true
  },
  "sandbox": {
    "cpus": 4,
    "memory_gb": 6,
    "pids": 256,
    "timeout_minutes": 30
  },
  "providers": {
    "codex": {
      "enabled": false,
      "status": "UNCONFIGURED",
      "model": null,
      "quota": null,
      "require_preflight": true
    },
    "claude": {
      "enabled": false,
      "status": "UNCONFIGURED",
      "model": null,
      "quota": null,
      "require_preflight": true
    },
    "antigravity": {
      "enabled": false,
      "status": "UNCONFIGURED",
      "model": null,
      "quota": null,
      "require_preflight": true
    }
  },
  "preferences": {
    "implementation": [
      "codex",
      "claude",
      "antigravity"
    ],
    "review": [
      "claude",
      "codex",
      "antigravity"
    ],
    "ui_qa": [
      "antigravity",
      "codex",
      "claude"
    ]
  },
  "merge_auto": false,
  "deploy_auto": false,
  "deployment": {
    "topology": "single-vps",
    "worker_location": "same-vps",
    "selected_profile": "bom",
    "profile_status": "proposta-nao-contratada",
    "cpu_vcpu": 8,
    "ram_gb": 16,
    "disk_gb": 200,
    "gpu_required": false
  },
  "admission": {
    "ram_percent_threshold": 85,
    "ram_sustained_minutes": 5,
    "minimum_free_disk_percent": 20
  }
}
```